SSH Honeypot Deployment using Cowrie

Author: Ajaya Jaiswal

What is a Honeypot?

A honeypot is a deliberately deployed computer system or network resource designed to attract unauthorized attackers. It mimics a real and valuable system so that attackers interact with it instead of actual production systems.

Purpose:
Honeypots are used to observe, track, and analyze attacker behavior, techniques, and tools. Security professionals study these interactions to better understand threats and improve defenses.

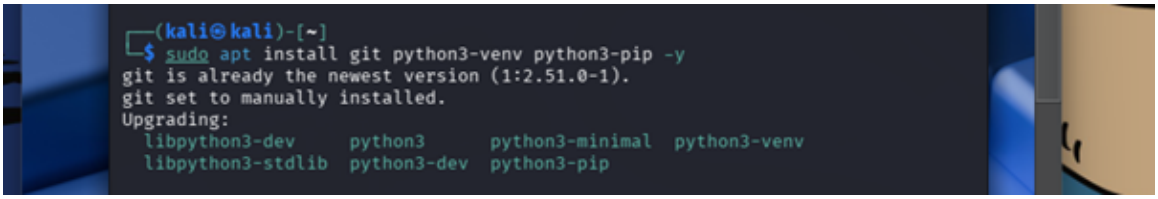
Design:
The system is built to look like a legitimate and valuable target, such as a database server, application server, or network service.

Realism:
To make the trap convincing, honeypots often contain fake data and intentional vulnerabilities that lure attackers into interacting with the system.

1. Honeypot Deployment Stage

In this stage I set up the Cowrie SSH honeypot on a Linux virtual machine. The purpose of this stage is to prepare the environment where the fake server will run and wait for attackers to connect.

In this screenshot I am preparing the honeypot environment. The commands shown in the terminal install required dependencies, clone the Cowrie repository, and configure the honeypot service. This step ensures that the honeypot server is ready to run and simulate a real SSH system.



```
┌──(kali㉿kali)-[~]
└─$ git clone https://github.com/cowrie/cowrie
Cloning into 'cowrie'...
remote: Enumerating objects: 21338, done.
remote: Counting objects: 100% (150/150), done.
remote: Compressing objects: 100% (99/99), done.
remote: Total 21338 (delta 92), reused 51 (delta 51), pack-reused 21188 (from
 2)
Receiving objects: 100% (21338/21338), 11.47 MiB | 13.76 MiB/s, done.
Resolving deltas: 100% (14831/14831), done.
```

```
┌──(kali㉿kali)-[~]
└─$ cd cowrie

┌──(kali㉿kali)-[~/cowrie]
└─$ python3 -m venv cowrie-env

┌──(kali㉿kali)-[~/cowrie]
└─$ source cowrie-env/bin/activate
```

```
┌──(cowrie-env)─(kali㉿kali)-[~/cowrie]
└─$ pip install --upgrade pip
Requirement already satisfied: pip in ./cowrie-env/lib/python3.13/site-packag
es (25.2)
Collecting pip
  Downloading pip-26.0.1-py3-none-any.whl.metadata (4.7 kB)
Downloading pip-26.0.1-py3-none-any.whl (1.8 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.8/1.8 MB 3.4 MB/s  0:00:00
Installing collected packages: pip
```

```
┌──(cowrie-env)─(kali㉿kali)-[~/cowrie]
└─$ pip install -r requirements.txt
Collecting attrs==25.4.0 (from -r requirements.txt (line 1))
  Downloading attrs-25.4.0-py3-none-any.whl.metadata (10 kB)
Collecting bcrypt==5.0.0 (from -r requirements.txt (line 2))
  Downloading bcrypt-5.0.0-cp39-abi3-manylinux_2_34_x86_64.whl.metadata (10 k
B)
```

```
┌──(cowrie-env)─(kali㉿kali)-[~/cowrie]
└─$ cp etc/cowrie.cfg.dist etc/cowrie.cfg
```

```
┌──(cowrie-env)─(kali㉿kali)-[~/cowrie]
└─$ twistd -n cowrie
Usage: twistd [options]
Options:
```

```
┌──(cowrie-env)─(kali㉿kali)-[~/cowrie]
└─$ pip install -e .
Obtaining file:///home/kali/cowrie
  Installing build dependencies ... done
  Checking if build backend supports build_editable ... done
  Getting requirements to build editable ... done
  Preparing editable metadata (pyproject.toml) ... done
Requirement already satisfied: attrs==25.4.0 in ./cowrie-env/lib/python3.13/s
ite-packages (from cowrie==2.9.14.dev12+ge3f1f7426) (25.4.0)
Requirement already satisfied: bcrypt==5.0.0 in ./cowrie-env/lib/python3.13/s
```

```
┌──(cowrie-env)─(kali㉿kali)-[~/cowrie]
└─$ ./cowrie-env/bin/cowrie start

Join the Cowrie community at: https://www.cowrie.org/slack/

Starting cowrie: [twistd --umask=0022 --pidfile /home/kali/cowrie/var/run/cow
rie.pid --logger cowrie.python.logfile.logger cowrie] ...
/home/kali/cowrie/cowrie-env/lib/python3.13/site-packages/twisted/conch/ssh/t
ransport.py:110: CryptographyDeprecationWarning: TripleDES has been moved to
cryptography.hazmat.decrepit.ciphers.algorithms.TripleDES and will be removed
 from cryptography.hazmat.primitives.ciphers.algorithms in 48.0.0.
```

```
┌──(cowrie-env)─(kali㉿kali)-[~/cowrie]
└─$ ./cowrie-env/bin/cowrie status
cowrie is running (PID: 19252).
```

2. Attacker Interaction Capture

After the honeypot is deployed, I simulate an attacker connecting to the server from another terminal. The goal of this stage is to demonstrate how attackers interact with the system and how Cowrie records their activity.

In this screenshot I connect to the honeypot using SSH. The attacker attempts to log in using a username and password. Cowrie presents a fake Linux shell environment to the attacker while secretly recording all actions.

```
┌──(kali㉿kali)-[~]
└─$ ssh root@localhost -p 2222
The authenticity of host '[localhost]:2222 ([127.0.0.1]:2222)' can't be estab
lished.
ED25519 key fingerprint is: SHA256:ifA4p/QG+6EzUjKINENRiH9b//M/80D2gGoJILI6no
Q
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
```

3. Command Logging

Once the attacker gains access to the fake shell environment, typical reconnaissance commands are executed. These commands help attackers gather information about the system. Cowrie records every command entered during the session.

This screenshot shows commands executed by the simulated attacker such as 'whoami', 'uname -a', and 'cat /etc/passwd'. These commands are commonly used to collect system information. Cowrie logs these commands so security analysts can study attacker behavior.

4. Log Analysis on Honeypot Terminal

In this stage I monitor the honeypot logs to observe attacker activity in real time. Cowrie stores detailed information about every connection attempt and command executed.

The honeypot logs show important security information including:
- Attacker IP address
- Username used during login
- Password attempts
- Commands executed in the session

These logs allow security analysts to understand attacker behavior and identify common attack patterns.

```
┌──(cowrie-env)─(kali㉿kali)-[~/cowrie]
└─$ tail -f var/log/cowrie/cowrie.log
2026-03-14T10:00:36.194270Z [HoneyPotSSHTransport,0,127.0.0.1] Command found:
 whoami
2026-03-14T10:00:49.230934Z [HoneyPotSSHTransport,0,127.0.0.1] CMD: uname -a
2026-03-14T10:00:49.231557Z [HoneyPotSSHTransport,0,127.0.0.1] Command found:
 uname -a
2026-03-14T10:01:14.274680Z [HoneyPotSSHTransport,0,127.0.0.1] CMD: cat /etc/
passwd
2026-03-14T10:01:14.275277Z [HoneyPotSSHTransport,0,127.0.0.1] Command found:
 cat /etc/passwd
2026-03-14T10:03:25.804471Z [-] Timeout reached in HoneyPotSSHTransport
2026-03-14T10:03:25.807089Z [HoneyPotSSHTransport,0,127.0.0.1] Closing TTY Lo
g: var/lib/cowrie/tty/39a476e2fc2c038c9647a394e9e3609cc15d2a8bfd382a144448650
48042cfaa after 300.0 seconds
2026-03-14T10:03:25.808049Z [HoneyPotSSHTransport,0,127.0.0.1] avatar root lo
gging out
2026-03-14T10:03:25.808214Z [cowrie.ssh.transport.HoneyPotSSHTransport#info]
connection lost
2026-03-14T10:03:25.808667Z [HoneyPotSSHTransport,0,127.0.0.1] Connection los
t after 380.5 seconds
^C
```

```
┌──(cowrie-env)─(kali㉿kali)-[~/cowrie]
└─$ cat var/log/cowrie/cowrie.json
{"eventid":"cowrie.session.connect","src_ip":"127.0.0.1","src_port":46320,"ds
t_ip":"127.0.0.1","dst_port":2222,"session":"93dc05ee572a","protocol":"ssh","
message":"New connection: 127.0.0.1:46320 (127.0.0.1:2222) [session: 93dc05ee
572a]","sensor":"kali","uuid":"31d8bd46-1fad-11f1-9750-08002763b005","timesta
mp":"2026-03-14T09:57:05.271511Z"}
{"eventid":"cowrie.client.version","version":"SSH-2.0-OpenSSH_10.2p1 Debian-2
","message":"Remote SSH version: SSH-2.0-OpenSSH_10.2p1 Debian-2","sensor":"k
ali","uuid":"31d8bd46-1fad-11f1-9750-08002763b005","timestamp":"2026-03-14T09
:57:05.272807Z","src_ip":"127.0.0.1","session":"93dc05ee572a","protocol":"ssh
"}
```

Project Summary

In this project I successfully deployed an SSH honeypot using the Cowrie framework. The honeypot simulated a vulnerable Linux server and captured attacker interactions. By connecting to the honeypot and executing commands, I demonstrated how Cowrie records login attempts, attacker commands, and session activities. The collected logs provide valuable insight into attacker behavior and can help security professionals study real-world attack techniques without risking actual production systems.

Acknowledgment

I used ChatGPT as a learning aid during this project. It helped me understand the formatting and structure of the report, improve the clarity of explanations, and troubleshoot issues during the installation and configuration of the Cowrie honeypot. ChatGPT also assisted in debugging commands and identifying errors when I encountered problems during the setup process. The tool was used for guidance and clarification, while the implementation, testing, and analysis of the honeypot were completed by me.